An Iterative Algorithm for Maximum Flow

1. The Maximum-Flow Problem
   - weighted directed graphs as flow networks
   - the Ford-Fulkerson algorithm
   - termination and running time
2. Maximum Flows and Minimum Cuts
   - running a numerical example
   - flows and cuts
   - max flow equals min cut
3. Choosing Good Augmenting Paths
   - an example with a large number of iterations
   - the scaling max-flow algorithm

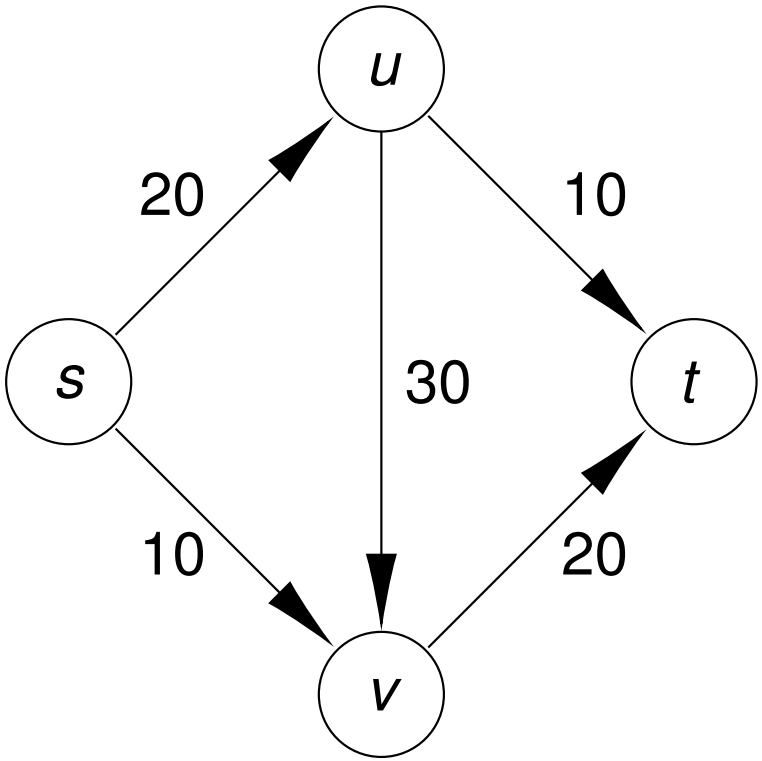
CS 401/MCS 401 Lecture 15
Computer Algorithms I
Jan Verschelde, 22 July 2026

An Iterative Algorithm for Maximum Flow

1. The Maximum-Flow Problem
   - weighted directed graphs as flow networks
   - the Ford-Fulkerson algorithm
   - termination and running time
2. Maximum Flows and Minimum Cuts
   - running a numerical example
   - flows and cuts
   - max flow equals min cut
3. Choosing Good Augmenting Paths
   - an example with a large number of iterations
   - the scaling max-flow algorithm

weighted directed graphs as flow networks

A flow network is defined by a weighted directed graph $G = (V, E)$:



- s is the only *source* node
- t is the only *sink* node
- each edge has a *capacity*
- nodes other than s or t are *internal* nodes

$V = \{ s, u, v, t \}$

$E = \{ (s, u, 20), (s, v, 10), (u, v, 30), (v, t, 20), (u, t, 10) \}$

A communication network structures the flow of information.

defining flow

Let $G = (V, E)$ be a weighted directed graph with a source s, sink t, and capacities c_e for each $e \in E$. An *s-t flow* is a function f

$$f : E \to \mathbb{R}^+ : e \mapsto f(e),$$

where $f(e)$ is the amount of flow carried by edge e.

Each flow must satisfy the following two properties:

1. *Capacity conditions:* for each $e \in E$: $f(e) \leq c_e$.
2. *Conservation conditions:* for each internal node v:

$$\sum_{e \text{ into } v} f(e) = \sum_{e \text{ out of } v} f(e)$$

By the capacity conditions, the flow carried by each edge cannot be larger than the capacity of the edge.
The conservation conditions state that internal nodes do not leak.

the value of a flow

The source has no incoming edges, but generates flow.
The sink has no outgoing edges, but absorbs flow.

Given a flow f, *the value $v(f)$ of the flow f* is $v(f) = \sum_{e \text{ out of } s} f(e)$.

We denote:

$$f^{\text{out}}(v) = \sum_{e \text{ out of } v} f(e) \text{ and } f^{\text{in}}(v) = \sum_{e \text{ into } v} f(e). \text{ Then : } v(f) = f^{\text{out}}(s).$$

The conservation laws for internal nodes v are $f^{\text{in}}(v) = f^{\text{out}}(v)$.
For sets $S \subseteq V$:

$$f^{\text{out}}(S) = \sum_{e \text{ out of } S} f(e) \quad \text{and} \quad f^{\text{in}}(S) = \sum_{e \text{ into } S} f(e).$$

the maximum flow problem

Given is $G = (V, E)$, a weighted directed graph with

- a single source $s \in V$,
- a single sink $t \in V$, and
- capacities c_e for each $e \in E$.

The *maximum flow problem* asks for an s-t flow f with the largest value $v(f)$ over all possible s-t flows.

As we solve the maximum flow problem, we will identify the edge that has the capacity which bounds the maximum flow.

uniqueness of the maximum flow problem

Exercise 1:

Is the solution to the maximum flow problem unique?

Justify your anwer with an example.

Formulate conditions on the input data for a solution to be or not to be unique.

multiple sources and sinks

Exercise 2:

Consider a network with two sources and two sinks.

Explain how to transform such network into a network with a single source and single sink.

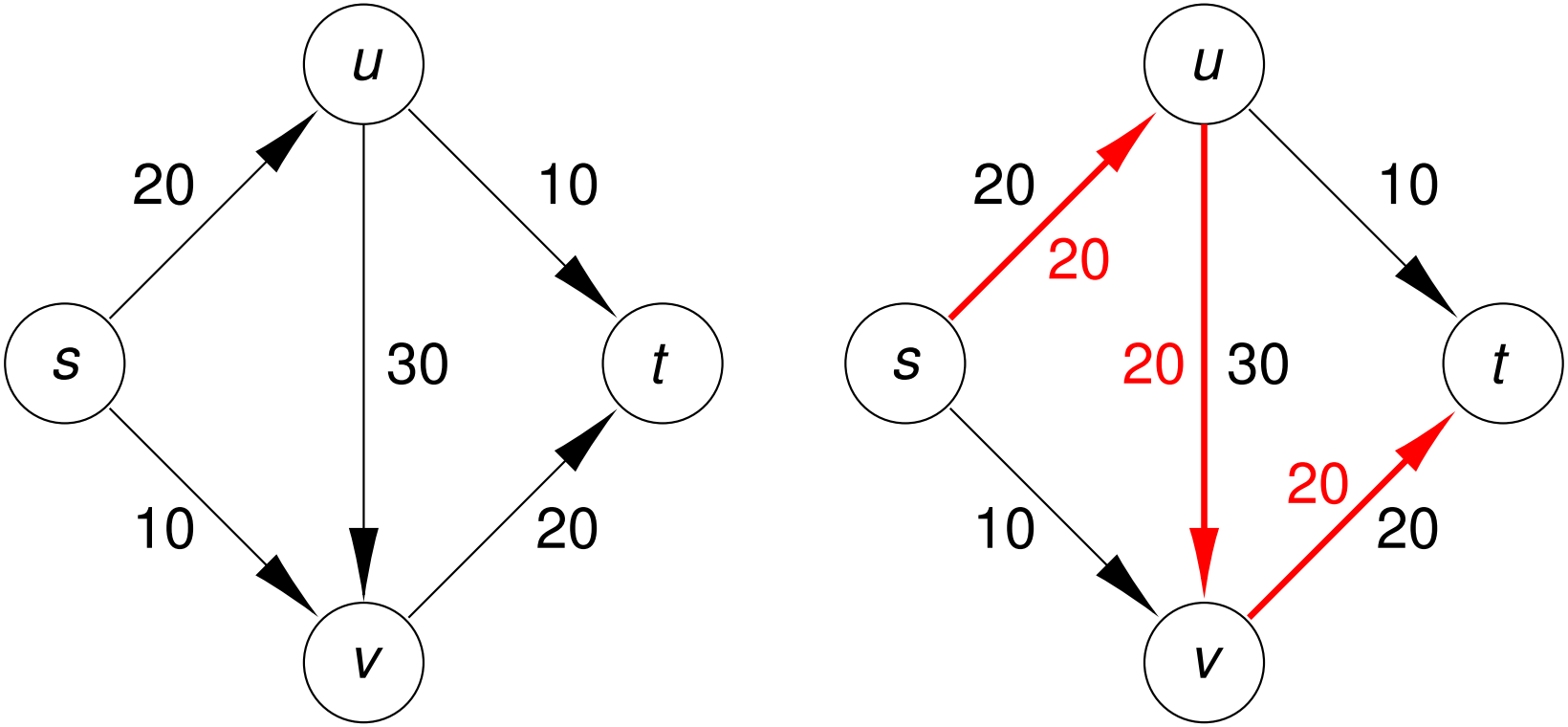
Illustrate your explanation with a good example.

An Iterative Algorithm for Maximum Flow

1. The Maximum-Flow Problem
   - weighted directed graphs as flow networks
   - the Ford-Fulkerson algorithm
   - termination and running time
2. Maximum Flows and Minimum Cuts
   - running a numerical example
   - flows and cuts
   - max flow equals min cut
3. Choosing Good Augmenting Paths
   - an example with a large number of iterations
   - the scaling max-flow algorithm

design of the algorithm

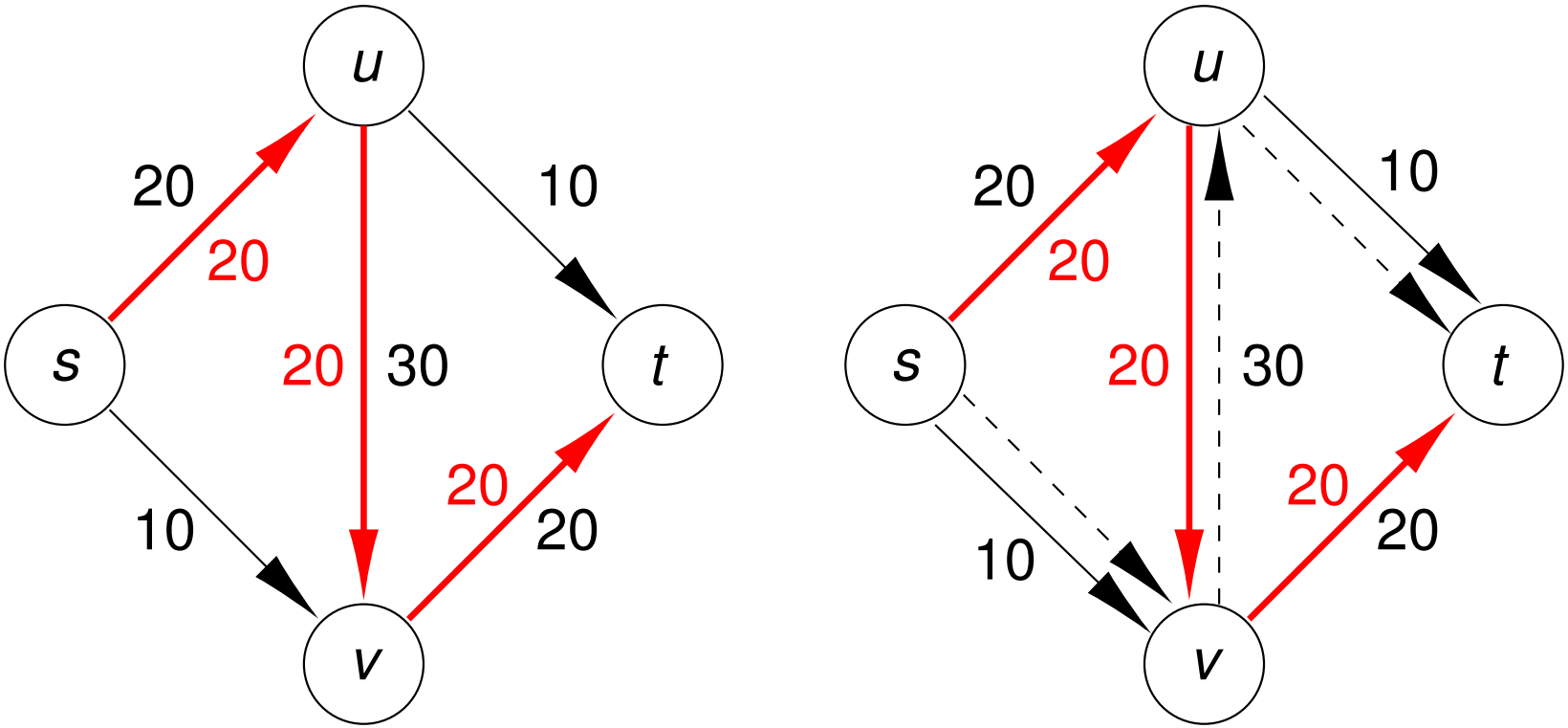
We push a flow of 20: $f((s,u)) = 20$, $f((u,v)) = 20$, $f((v,t)) = 20$:



Is this the maximum possible flow for this network?

computing an augmenting path

Given the flow $f((s,u)) = 20$, $f((u,v)) = 20$, $f((v,t)) = 20$:



We want to undo 10 units of flow from u to v.

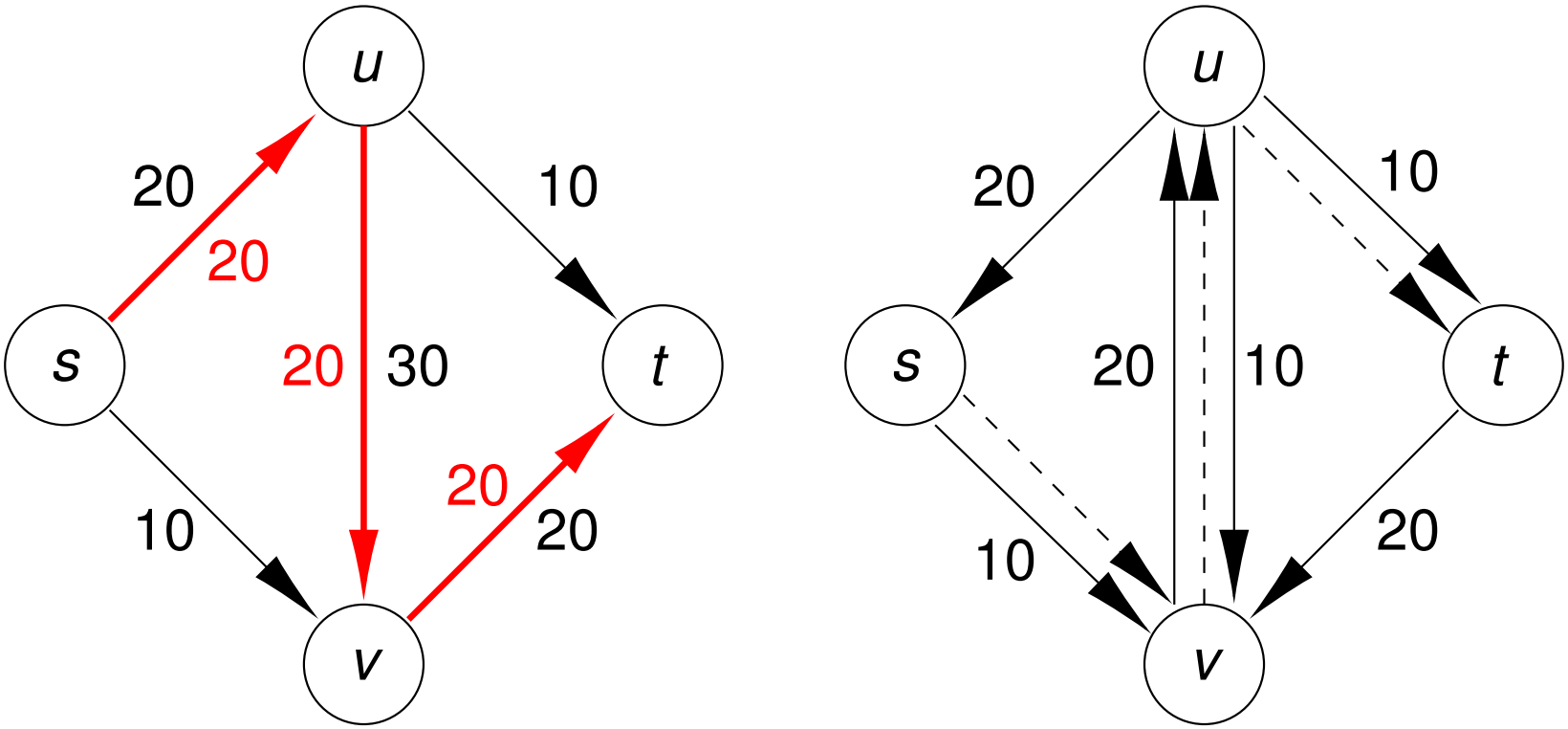
the residual graph

Given a network $G = (V, E)$ with flow f on G,
the *residual graph* $G_f = (V_f, E_f)$ of G with respect to f

- has the same vertices as G: $V_f = V$,
- for $e \in E$: $f(e) < c_e$, the forward edge $(u, v) \in E_f$ has capacity $c_e - f(e)$,
- for $e \in E$: $f(e) > 0$, the backward edge $(v, u) \in E_f$ has capacity $f(e)$.

Note: $\#E_f \leq 2\#E$.

a residual graph



The dashed arrows define the augmenting path.

a data structure question

Exercise 3:

Give a bound on the size of the residual graph in the size of G.

What data structure works best to store the residual graph?

residual capacity

Definition (1)

Given a flow f on a graph $G = (V, E)$, for $u, v \in V$,
the residual capacity of (u, v) is

$$c_f(u,v) = \begin{cases} c_{(u,v)} - f((u,v)) & \text{if } (u,v) \in E, \\ f((v,u)) & \text{if } (v,u) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

Example: Let $c_{(u,v)} = 16$ and $f((u,v)) = 11$.

1. We can increase $f((u,v))$ up to $c_f((u,v)) = 5$.
2. We allow to return up to 11 units of flow from v to u, and hence: $c_f((v,u)) = 11$.

the bottleneck

For any pair (u, v) of vertices, the residual capacity is

$$c_f(u,v) = \begin{cases} c_{(u,v)} - f((u,v)) & \text{if } (u,v) \in E, \\ f((v,u)) & \text{if } (v,u) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

Definition (2)

Given an s-t path P in G_f without repeat nodes,
the `bottleneck`(P, f) is the minimum residual capacity of any edge on P with respect to the flow f, computed as

$$\texttt{bottleneck}(P,f) = \min\{c_f(u,v) \mid (u,v) \in P\}.$$

As `bottleneck`$(P, f) > 0$, P is an *augmenting path*.

the operation `augment`(f, P)

`augment`(f, P)

Input: f, the flow on a network G,
P, an s-t path without repeat nodes.

Output: returns a new flow f'.

$b :=$ `bottleneck`(P, f)
$f' := f$
for each edge $(u, v) \in P$ do
  if (u, v) is a forward edge in G_f then
    $f'((u, v)) := f'((u, v)) + b$
  else (*we have a backward edge*)
    $f'((v, u)) := f'((v, u)) - b$
return f'

Any s-t path in G_f is an *augmenting path*.

Lemma (3)

$f' = \texttt{augment}(f, P)$ is a flow.

Proof. We verify the capacity and conservation conditions on f'.
It suffices to consider only the edges on P, as the rest of G was not changed by the operation `augment`.
Let $b = \texttt{bottleneck}(P, f)$. Consder e an edge on P.

1. Either e is a forward edge in G_f:

$$f'(e) = f(e) + b \leq f(e) + (c_e - f(e)) = c_e,$$

   so the capacity condition $f'(e) \leq c_e$ holds.

2. Or e is a backward edge in G_f, let $e = (u, v)$.

$$c_{(v,u)} \geq f((v, u)) \geq f'((v, u)) = f((v, u)) - b,$$

   As $b \leq f((v, u))$ and $b \geq 0$: $f'((v, u)) \leq c_{(v,u)}$.

proof continued: the conservation conditions on f'

On an internal node v, we have $f^{\text{in}}(v) = f^{\text{out}}(v)$,
because f is a flow and satifies the conservation conditions.

We must show that $(f')^{\text{in}}(v) = (f')^{\text{out}}(v)$.

Let e be a forward edge, $b = \texttt{bottleneck}(P, f)$, then:

$$(f')^{\text{in}}(v) = \sum_{e \text{ into } v} f'(e) = \sum_{e \text{ into } v} f(e) + b = f^{\text{in}}(v) + \sum_{e \text{ into } v} b,$$

and

$$(f')^{\text{out}}(v) = \sum_{e \text{ out of } v} f'(e) = \sum_{e \text{ out of } v} f(e) + b = f^{\text{out}}(v) + \sum_{e \text{ out of } v} b.$$

We see that $f^{\text{in}}(v) = f^{\text{out}}(v)$ implies $(f')^{\text{in}}(v) = (f')^{\text{out}}(v)$.

Verifying for backward edges is left to the next exercise.

Q.E.D.

verifying conservation conditions for backward edges

Exercise 4:
In the proof of Lemma (3) for an edge e on P,
verify the conservation conditions for backward edges.

the Ford-Fulkerson algorithm

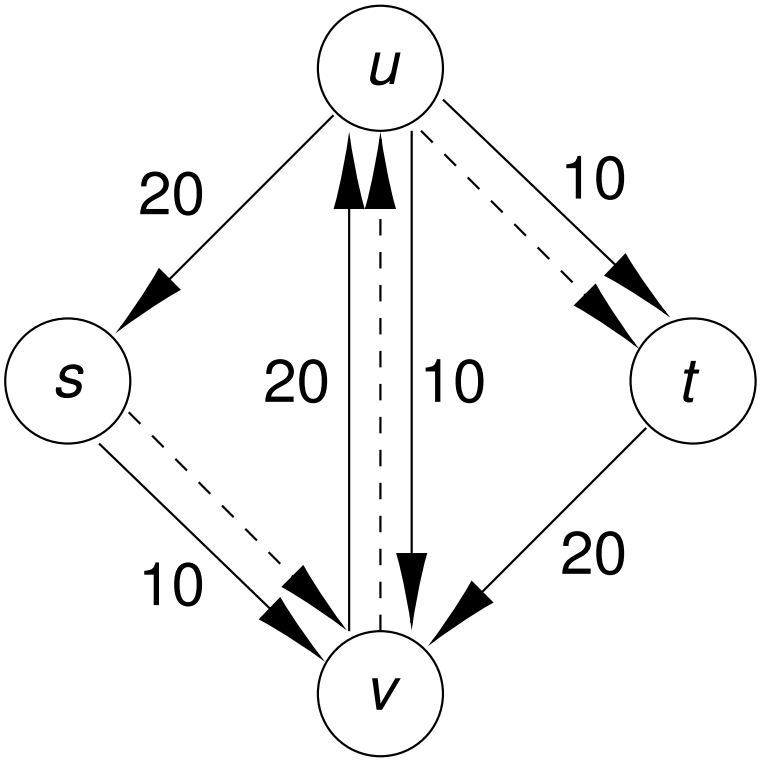
`Max-Flow`(G)

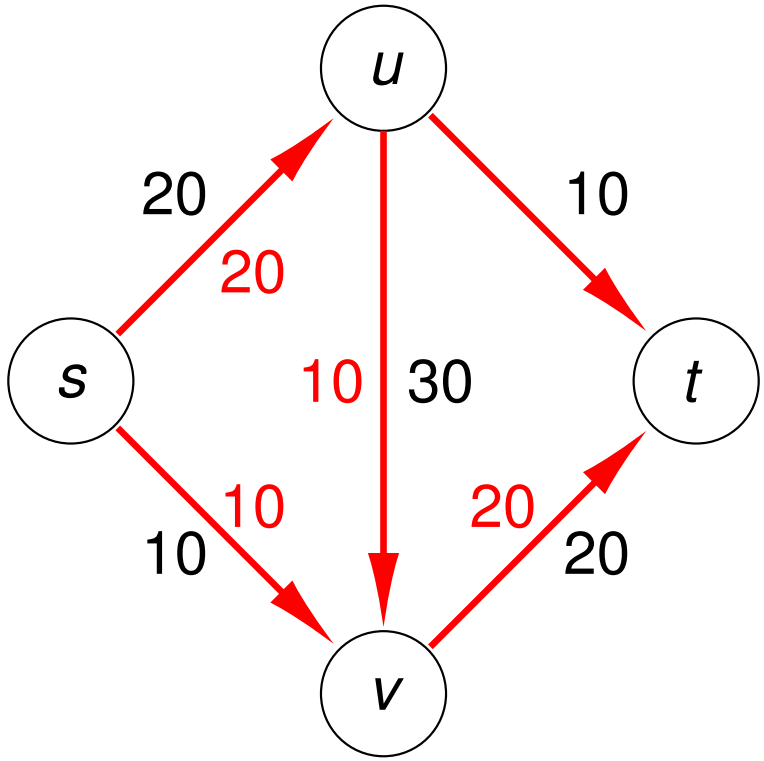
Input: $G = (V, E)$, a flow network.
Output: the maximum flow f.

for all $e \in E$: $f(e) := 0$
while there is an s-t path in G_f do
    let P be an s-t path with no repeat nodes in G_f
    $f := \texttt{augment}(f, P)$
return f

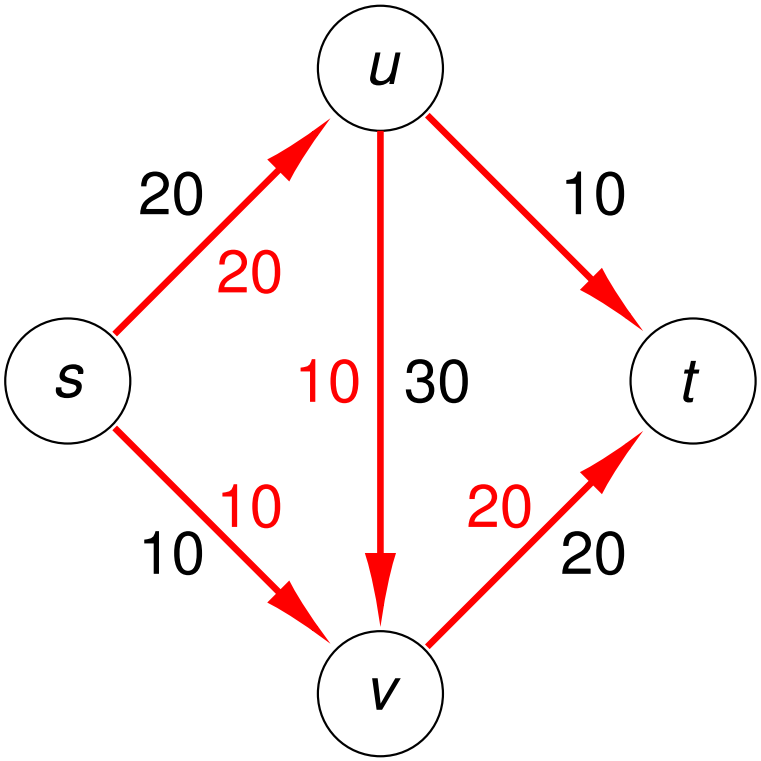
Does this iterative algorithm terminate?

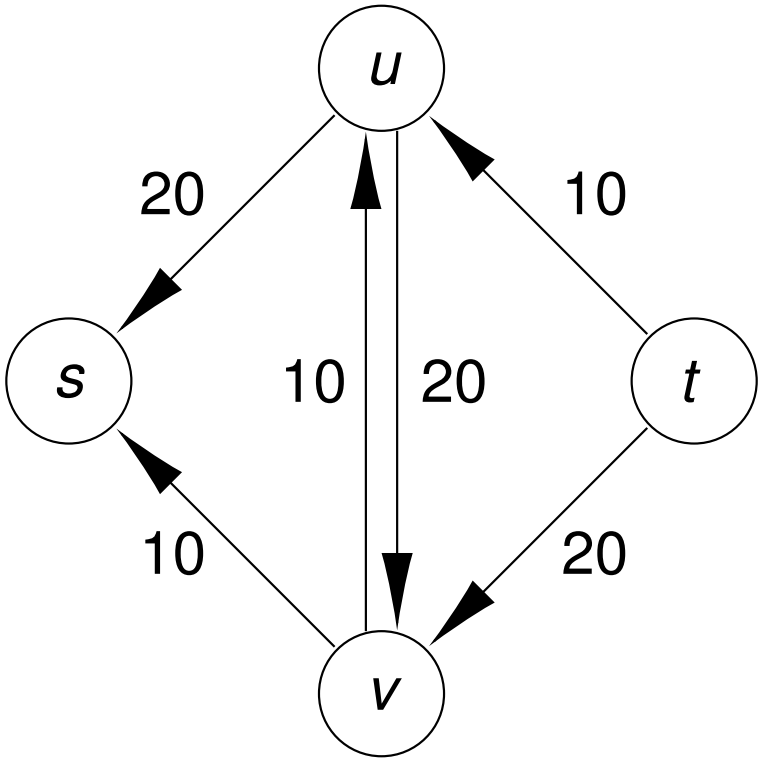
augmenting the flow





the residual graph





An Iterative Algorithm for Maximum Flow

1. The Maximum-Flow Problem
   - weighted directed graphs as flow networks
   - the Ford-Fulkerson algorithm
   - termination and running time
2. Maximum Flows and Minimum Cuts
   - running a numerical example
   - flows and cuts
   - max flow equals min cut
3. Choosing Good Augmenting Paths
   - an example with a large number of iterations
   - the scaling max-flow algorithm

integer capacities imply integer flows

Lemma (4)

If G has nonnegative integer values for each capacity, then at any stage of `Max-Flow`*, the values* $f(e)$ *and the residual capacities in* G_f *are nonnegative integers.*

Proof. We proceed by induction. Initially $f(e) = 0$ for all edges e.

Induction hypothesis: for all up to the k-th stage in `Max-Flow`, all $f(e)$ and residual capacities are nonnegative integer numbers. We show that the lemma holds for stage $k + 1$.

The value for `bottleneck`(P, f) is a nonnegative integer because, by the induction hypothesis, all residual capacities are nonnegative integers.

Thus `augment`(f, P) will return a flow with nonnegative integer values and also at stage $k + 1$ does the statement of the lemma hold.

Q.E.D.

the flow value strictly increases

Lemma (5)

Let f be a flow in G and P be an s-t path with no repeat nodes in G_f. If $\texttt{bottleneck}(P, f) > 0$, then $v(f') > v(f)$ for $f' = \texttt{augment}(f, P)$.

Proof. As $\texttt{bottleneck}(P, f) > 0$,
the first edge e on P in G_f must be out of s.
As P has no repeat nodes, P does not visit s again.
In a flow network G, the source s has no edges entering s.
Therefore, this first edge e must be a forward edge.

The flow on e is increased by $\texttt{bottleneck}(P, f)$
and the flow on other edges incident to s does not change.

Therefore, $v(f') = v(f) + \texttt{bottleneck}(P, f)$.

Q.E.D.

the algorithm terminates

Define the upper bound: $C = \sum_{e \text{ out of } s} c_e$.

Theorem (6)

If all capacities in the flow network are nonnegative integers, then the while loop in `Max-Flow` *terminates in at most C iterations.*

Proof. For all s-t flows f, we have $v(f) \leq C$.

By Lemma (5), the value of the flow increases in each step.

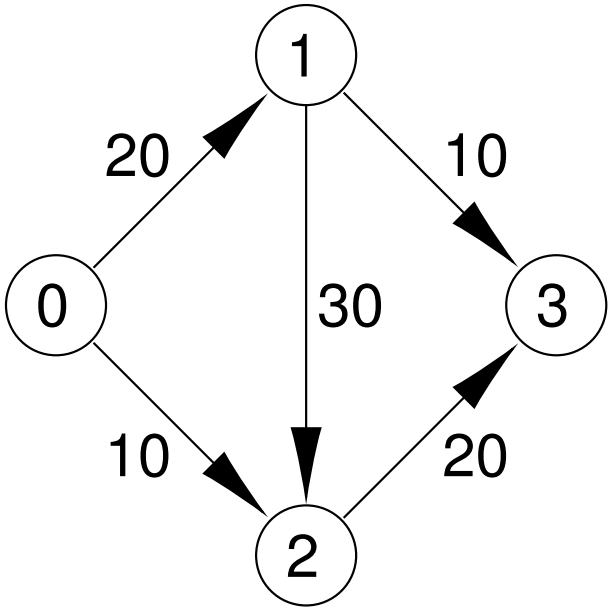
By Lemma (4), all flow values are nonnegative integers, so the increase is at least one in each iteration.

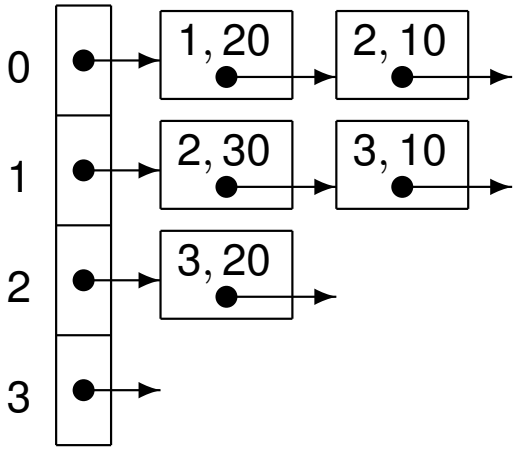
As the flow values start at zero and cannot go higher than C, the while loop runs for at most C iterations.

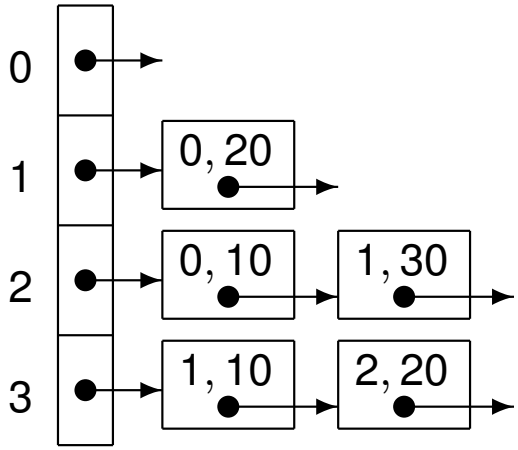
Q.E.D.

adjacency list representation

of a weighted directed graph







For every vertex,
we have two lists:

(1) vertices *to which* it has edges

(2) vertices *from which* it has edges

the running time

Assume $G = (V, E)$ is connected, then $m \geq n - 1$, $m = \#E$, $n = \#V$.

Theorem (7)

If all capacities on the m edges in the flow network are nonnegative integers, then `Max-Flow` *can be implemented to run in* $O(mC)$ *time.*

Proof. By Theorem (6), `Max-Flow` does at most C iterations.
We consider the work in each iteration.

1. G_f has at most $2m$ edges and is stored in an adjacency list representation. To find an s-t path, breadth-first or depth-first search runs in $O(m + n)$ time.
   By the assumption $m \geq n - 1$, $O(m + n)$ is $O(m)$.
2. The cost of `augment`(f, P) is $O(n)$ as the path P has no repeat nodes and thus at most $n - 1$ edges.

proof of the running time continued

3. Given the new flow computed by `augment`(f, P),
   the construction of G_f runs in $O(m)$:
   for each edge of G,
   we construct the forward and backward edges of G_f.

The number of operations per iteration is bounded by $O(m)$.

Since we have at most C iterations,
`Max-Flow` runs in $O(mC)$ time.

Q.E.D.

pseudo polynomial and strongly polynomial time

Recall that the subset sum and knapsack problems could be solved in $O(nW)$ time for n weights with upper bound W on their sum.

The Ford-Fulkerson algorithm has a similar running time: $O(mC)$ for a network with m edges and C the sum of all capacities.

Algorithms with a cost polynomial in the dimensions of the input and in the size of the numbers in the input data are called *pseudo polynomial*.

Algorithms with a cost polynomial in the dimensions of the input and with numbers which size is polynomial in the number of bits are called *strongly polynomial*.

An Iterative Algorithm for Maximum Flow

1. The Maximum-Flow Problem
   - weighted directed graphs as flow networks
   - the Ford-Fulkerson algorithm
   - termination and running time
2. Maximum Flows and Minimum Cuts
   - running a numerical example
   - flows and cuts
   - max flow equals min cut
3. Choosing Good Augmenting Paths
   - an example with a large number of iterations
   - the scaling max-flow algorithm

CS 401/MCS 401 Lecture 15
Computer Algorithms I
Jan Verschelde, 22 July 2026

An Iterative Algorithm for Maximum Flow

1. The Maximum-Flow Problem
   - weighted directed graphs as flow networks
   - the Ford-Fulkerson algorithm
   - termination and running time

2. Maximum Flows and Minimum Cuts
   - running a numerical example
   - flows and cuts
   - max flow equals min cut

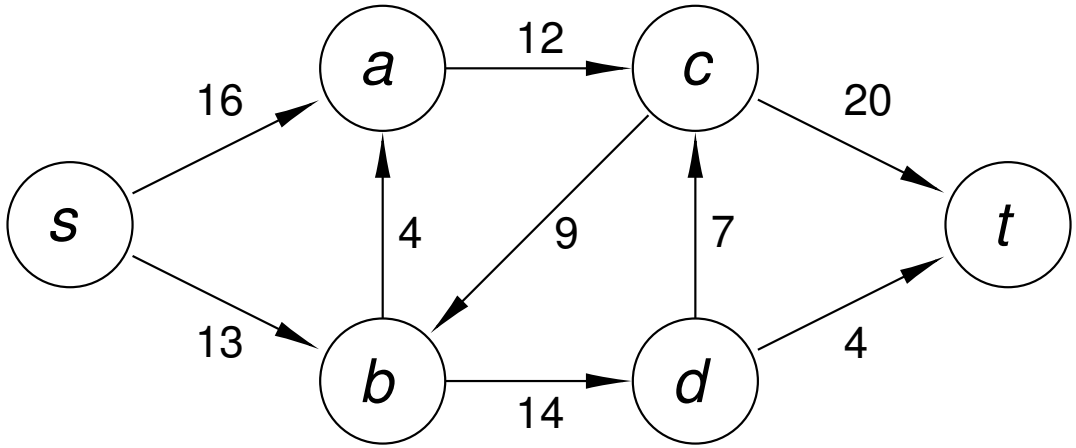
3. Choosing Good Augmenting Paths
   - an example with a large number of iterations
   - the scaling max-flow algorithm

a numerical example

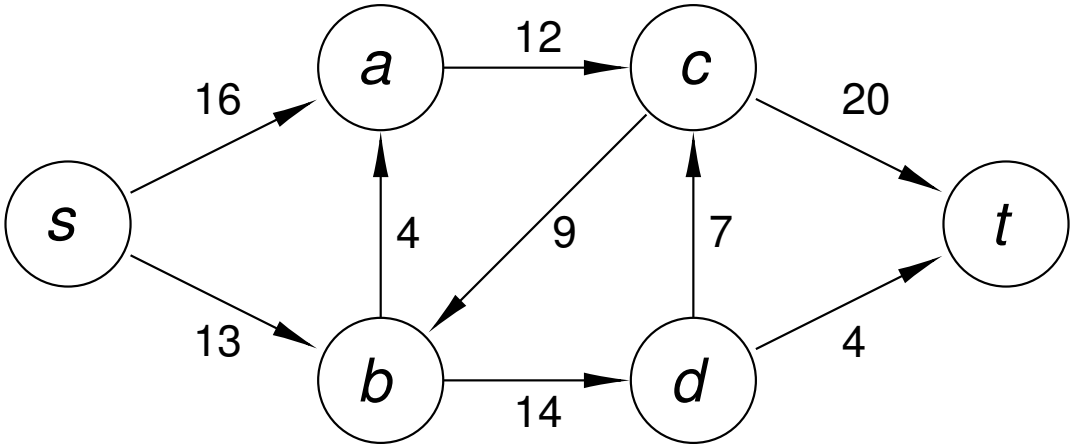
Let $G = (V, E)$ be the weighted directed graph with

- vertices $V = \{s, a, b, c, d, t\}$, and
- edges $E = \{(s, a, 16), (s, b, 13), (a, c, 12), (b, a, 4), (c, b, 9), (b, d, 14), (d, c, 7), (c, t, 20), (d, t, 4)\}$.

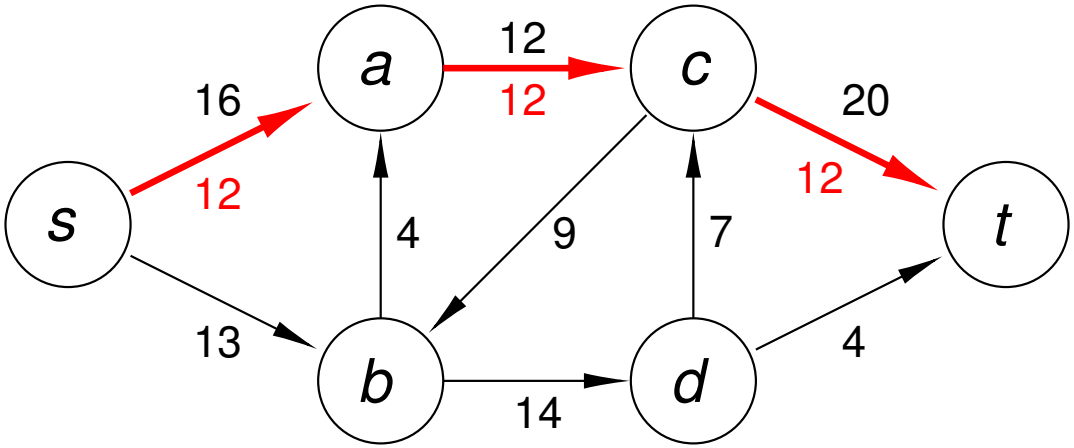
Compute the maximum flow from the source s to the sink t.



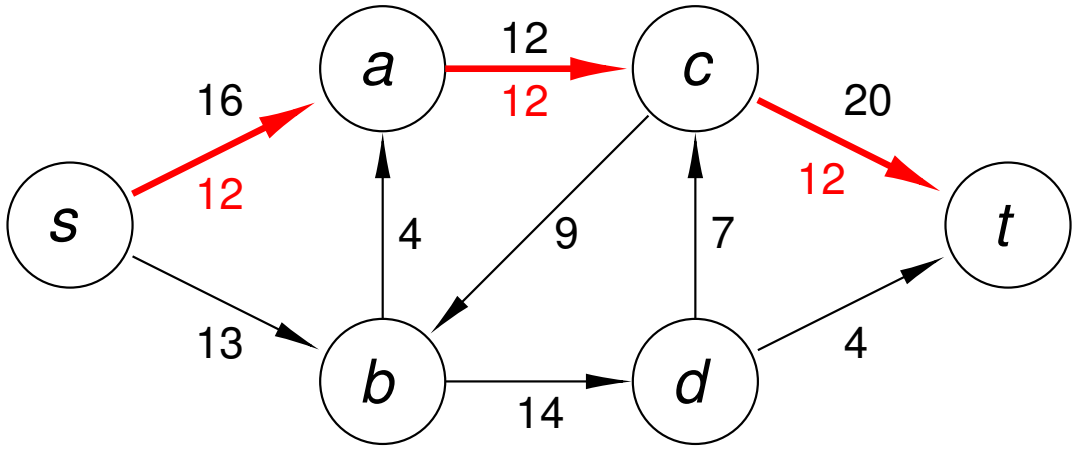
pushing a flow



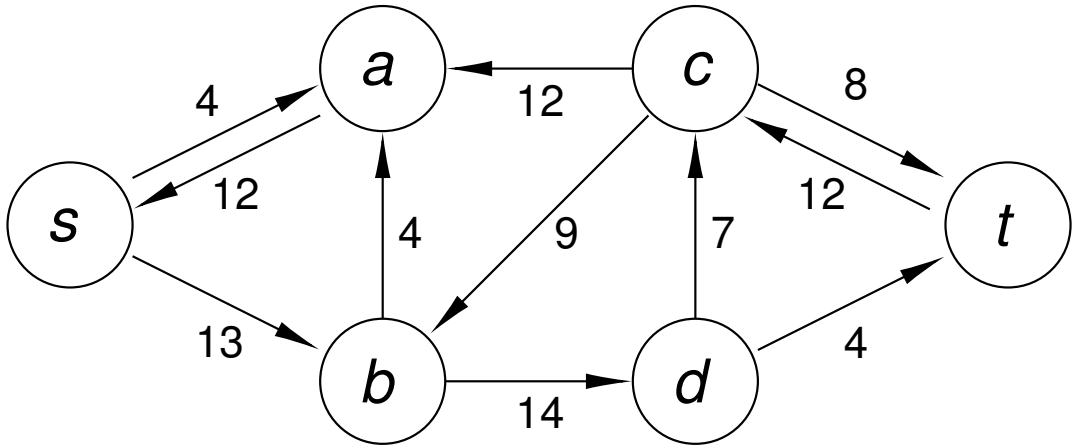
Step 1. We find a path from s to t and push a flow of 12.



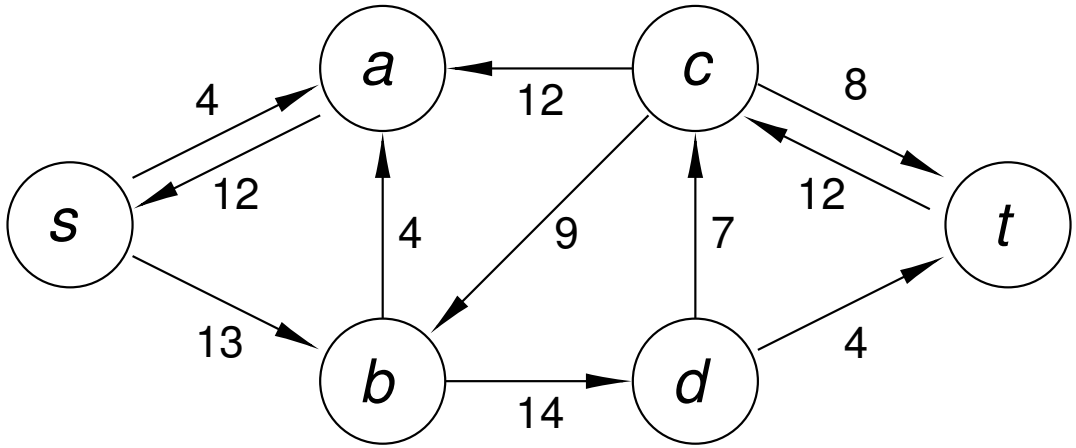
the residual graph



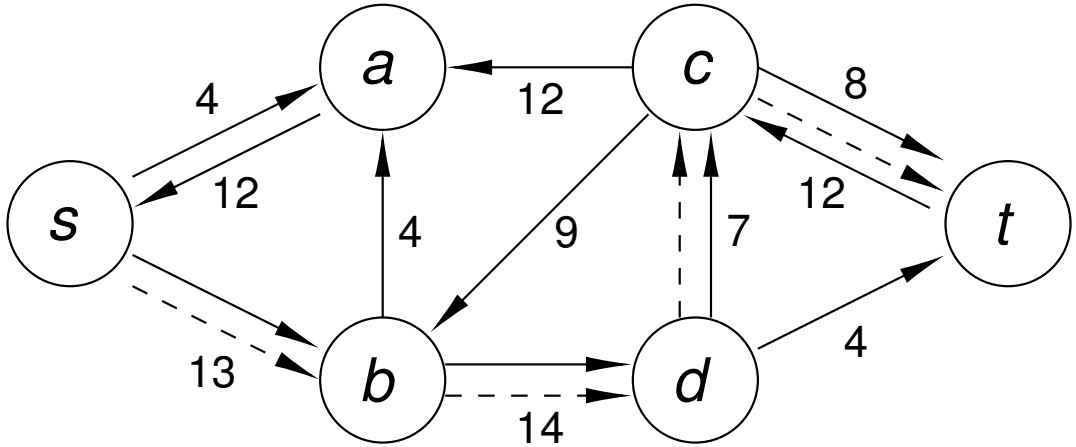
Step 2. Compute the residual graph.



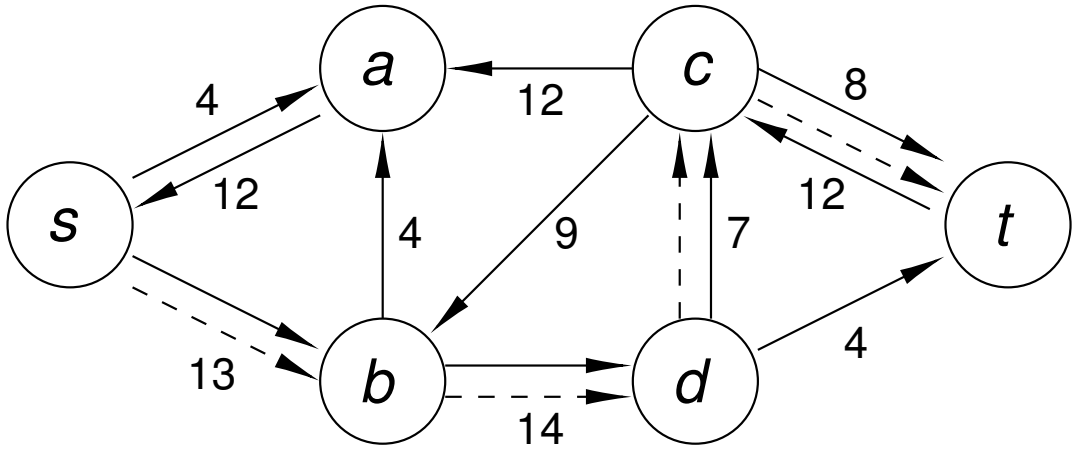
computing an augmenting path



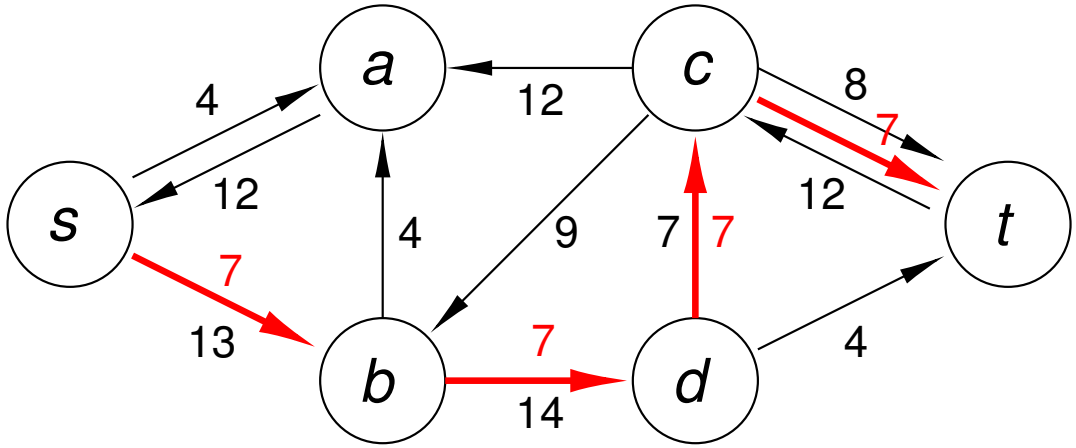
Step 2. Find a path from source s to sink t in the residual graph.



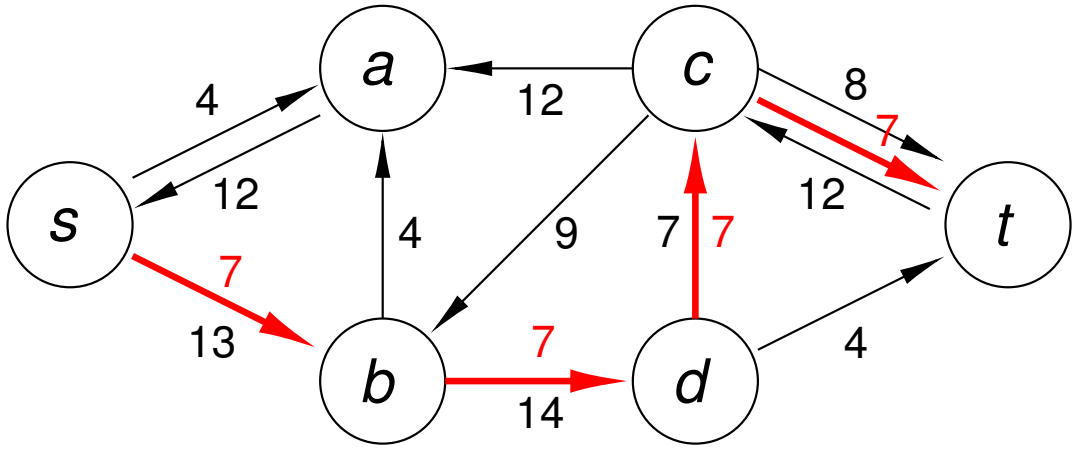
pushing a flow



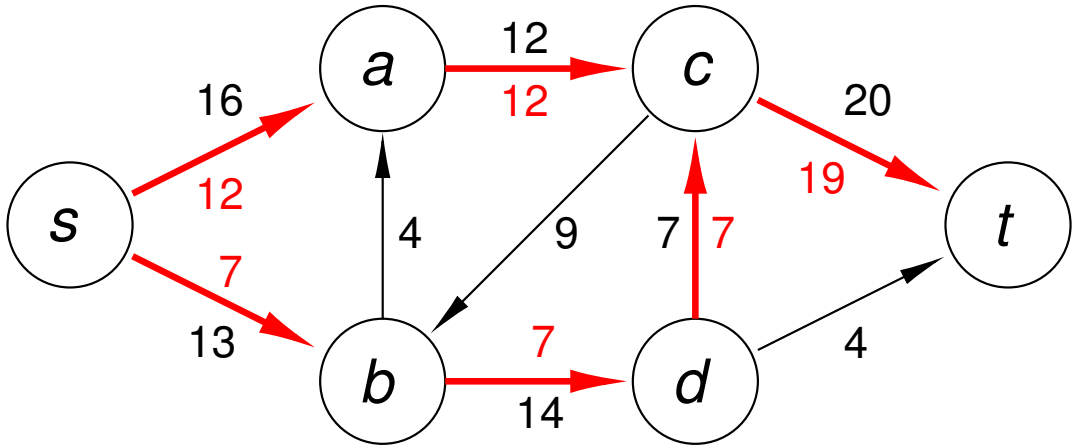
Step 2. The bottleneck is 7. Pushing a flow of 7.



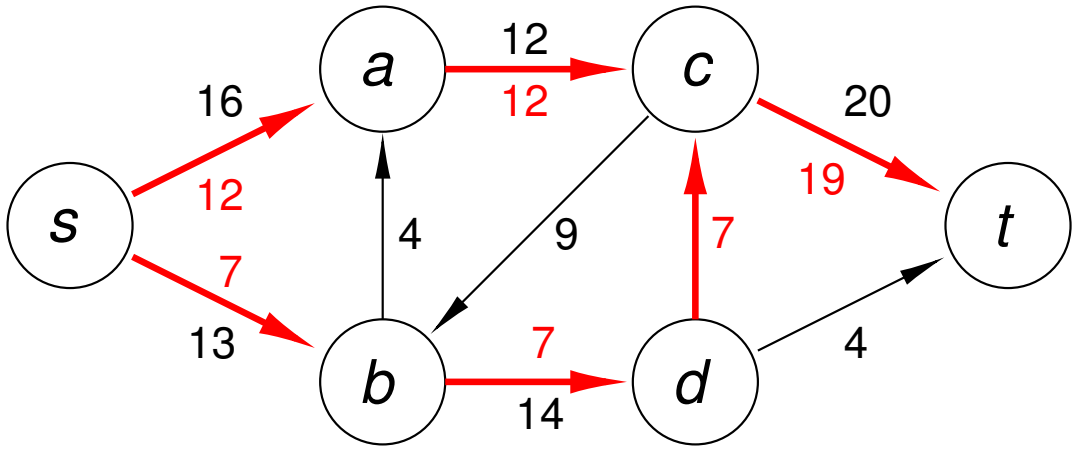
after two steps



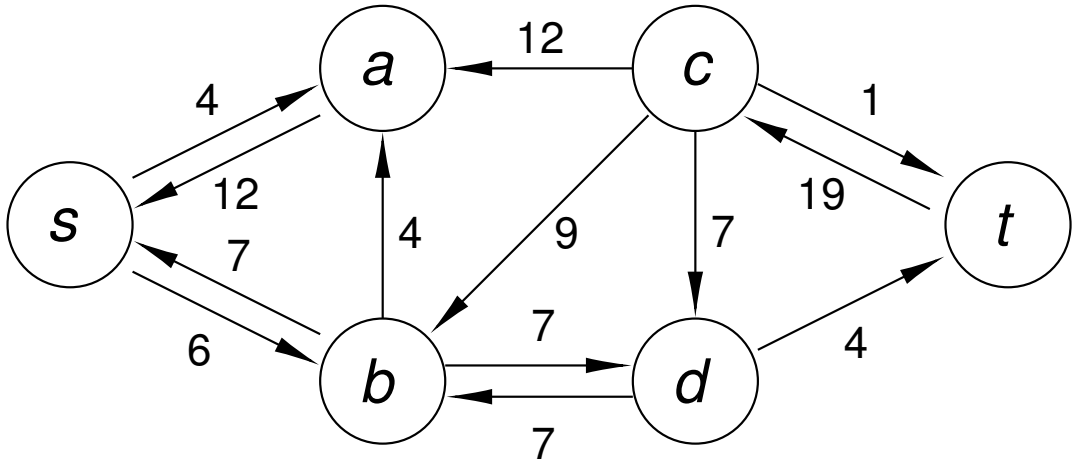
Step 2. After pushing flows of 12 and 7.



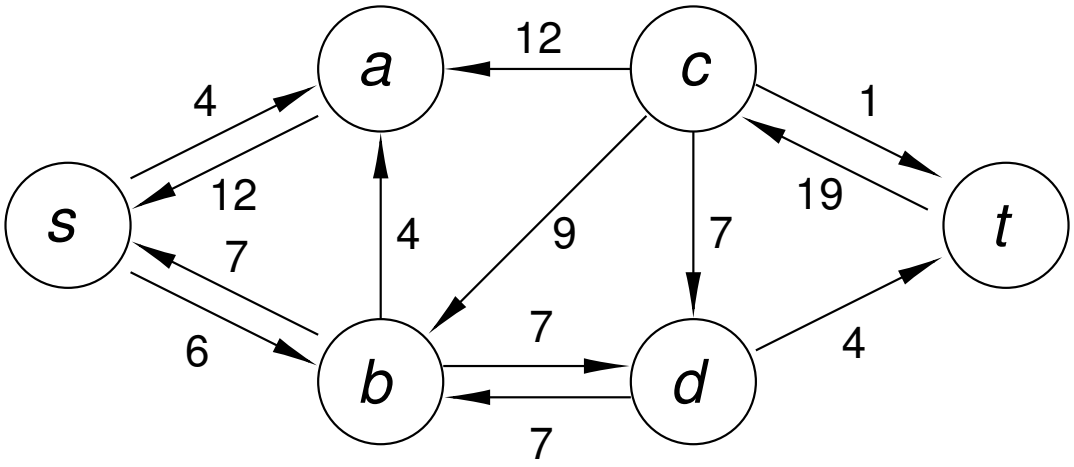
the residual graph



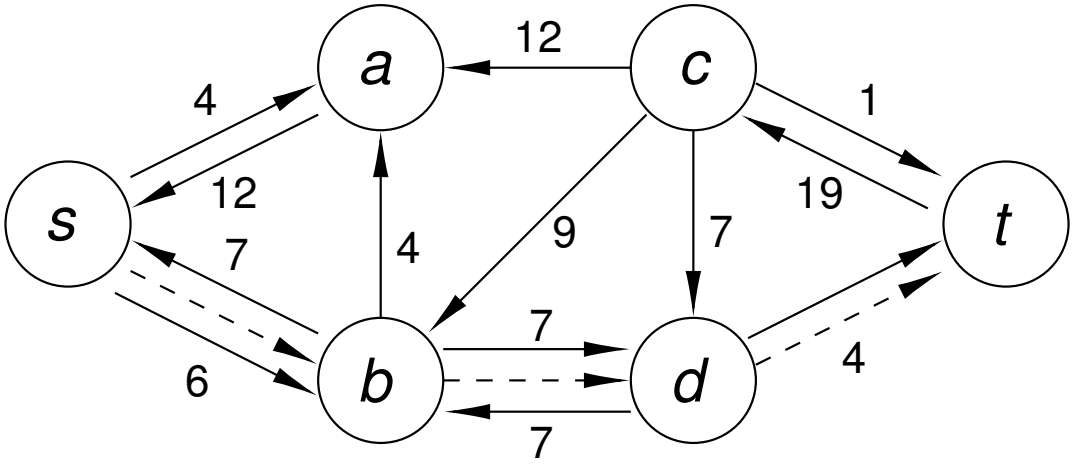
Step 3. Compute the residual graph.



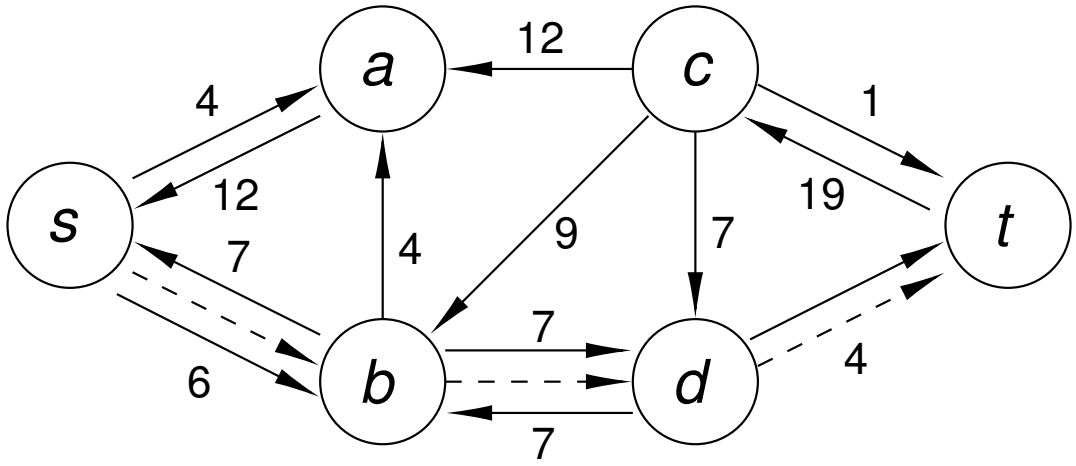
computing an augmenting path



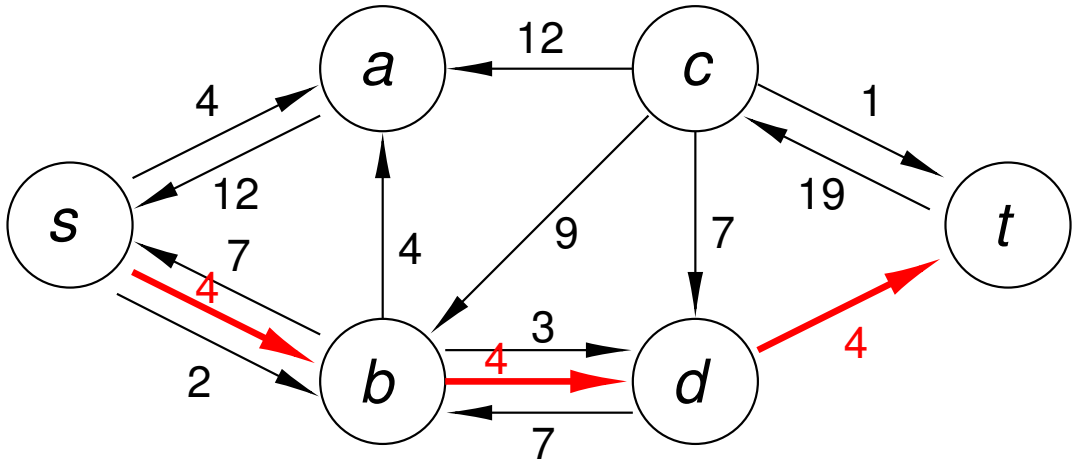
Step 3. Find a path from source s to sink t in the residual graph.



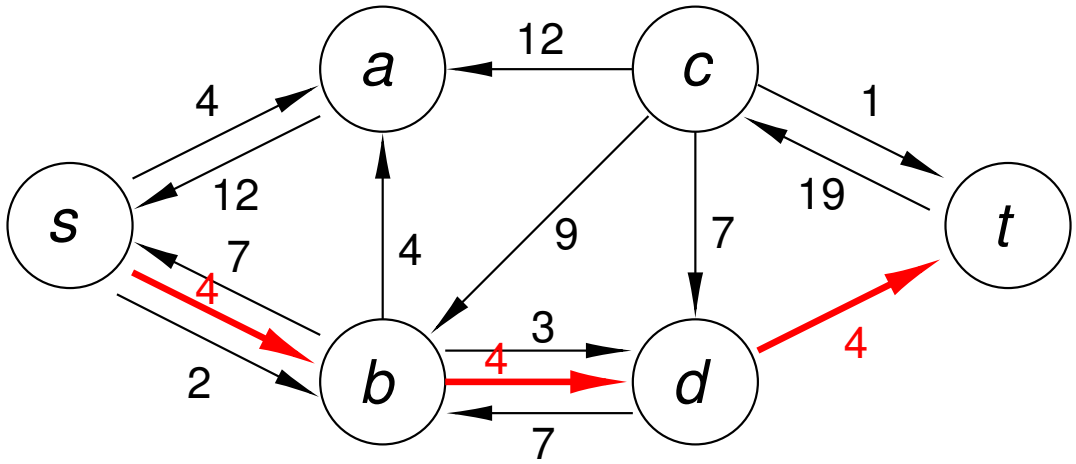
pushing a flow



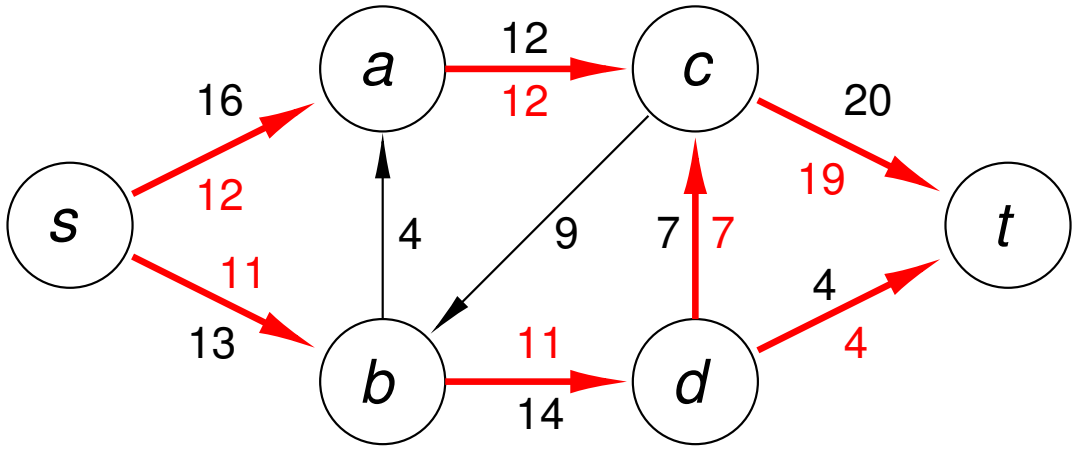
Step 3. The bottleneck is 4. Pushing a flow of 4.



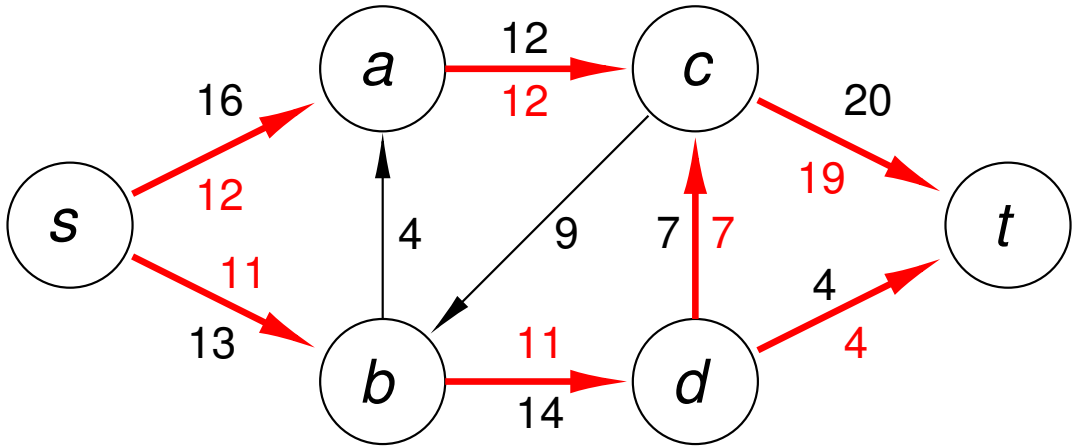
after three steps



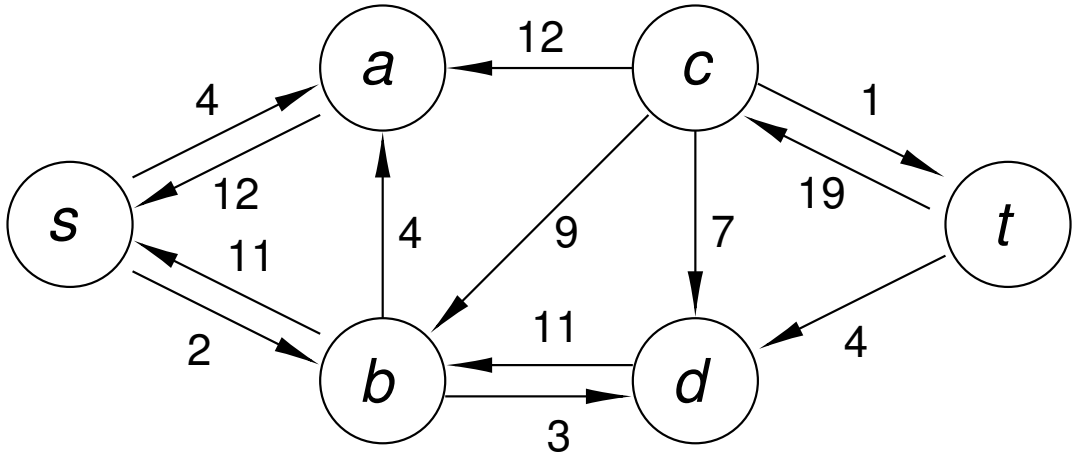
Step 4. After pushing flows of 12, 7, and 4.



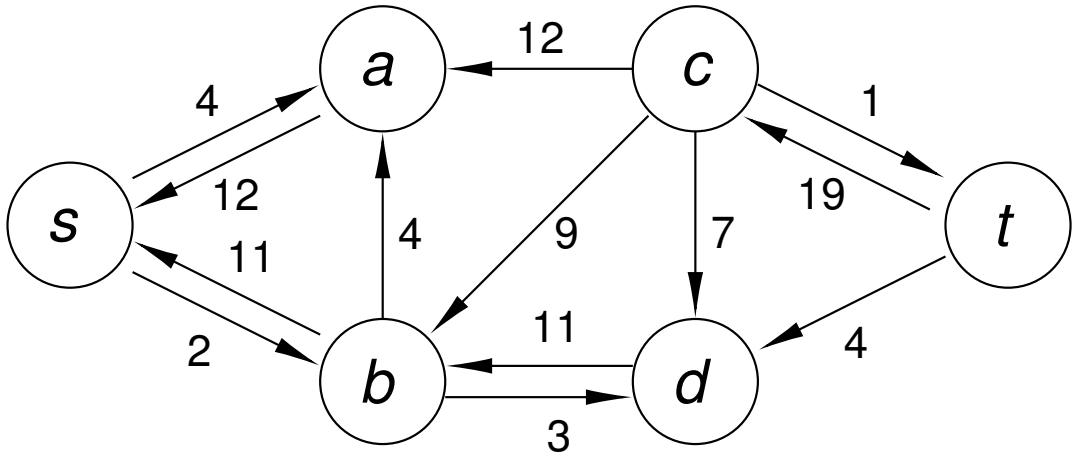
the residual graph



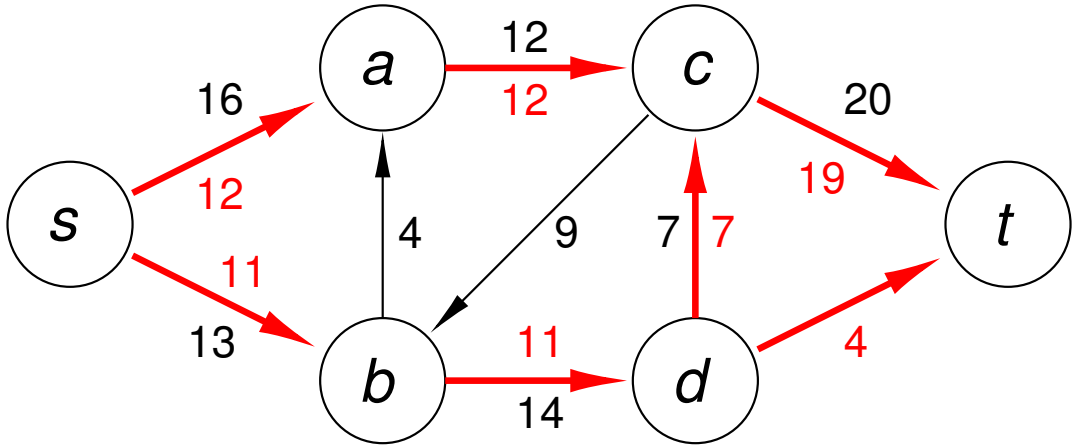
Step 4. Compute the residual graph.



computing an augmenting path



Step 4. No path in residual graph from source s to sink t exists. Done!



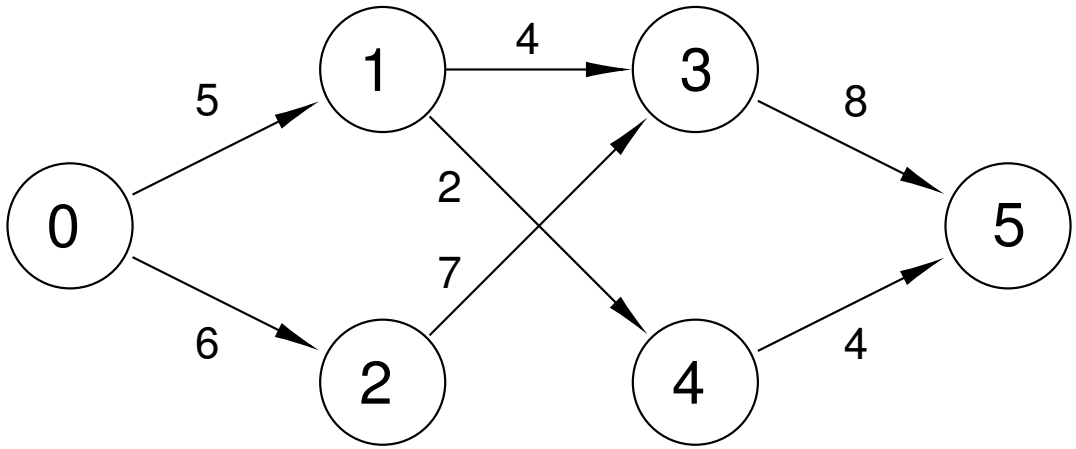
run a numerical example

Exercise 5:

Let $G = (V, E)$ be the weighted directed graph with

- vertices $V = \{0, 1, 2, 3, 4, 5\}$, and
- edges $E = \{(0, 1, 5), (0, 2, 6), (1, 3, 4), (1, 4, 2), (2, 3, 7), (3, 5, 8), (4, 5, 4)\}$.

Compute the maximum flow from the source 0 to the sink 5.



An Iterative Algorithm for Maximum Flow

1. The Maximum-Flow Problem
   - weighted directed graphs as flow networks
   - the Ford-Fulkerson algorithm
   - termination and running time
2. Maximum Flows and Minimum Cuts
   - running a numerical example
   - flows and cuts
   - max flow equals min cut
3. Choosing Good Augmenting Paths
   - an example with a large number of iterations
   - the scaling max-flow algorithm

flows and cuts

The bound $C = \sum_{e \text{ out of } s} c_e$ on $v(f)$ is often too high.

Definition (8)

Given a flow network $G = (V, E)$,
an s-t cut is a partition of V into A and B such that $s \in A$ and $t \in B$, denoted by $\text{cut}(A, B)$.

Definition (9)

Given a flow network $G = (V, E)$, *the capacity of* $\text{cut}(A, B)$ is

$$c(A, B) = \sum_{e \text{ out of } A} c_e.$$

value of a flow and flow across a cut

Lemma (10)

For any s-t flow f and any $\mathrm{cut}(A, B)$*, an s-t cut:* $v(f) = f^{\mathrm{out}}(A) - f^{\mathrm{in}}(A)$.

Proof. By definition, $v(f) = f^{\mathrm{out}}(s)$ and $f^{\mathrm{in}}(s) = 0$ as the source s has no incoming edges. Thus: $v(f) = f^{\mathrm{out}}(s) - f^{\mathrm{in}}(s)$. By the conservation conditions, for all internal nodes v, $f^{\mathrm{out}}(v) - f^{\mathrm{in}}(v) = 0$. Thus,

$$v(f) = \sum_{v \in A} \left(f^{\mathrm{out}}(v) - f^{\mathrm{in}}(v) \right).$$

If an edge e has both its vertices in A, then $f(e)$ appears once in the positive and once in the negative part of the sum above.
So we may restrict the sum to the other edges:

$$\sum_{v \in A} \left(f^{\mathrm{out}}(v) - f^{\mathrm{in}}(v) \right) = \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ into } A} f(e) = f^{\mathrm{out}}(A) - f^{\mathrm{in}}(A).$$

Q.E.D.

a bound on the value of the flow

Since a cut is a partition of the vertex set:

- edges out of A go into B, so $f^{\text{out}}(A) = f^{\text{in}}(B)$, and
- edges into A go out of B, so $f^{\text{in}}(A) = f^{\text{out}}(B)$.

So we can rewrite $v(f) = f^{\text{out}}(A) - f^{\text{in}}(A)$ as $v(f) = f^{\text{in}}(B) - f^{\text{out}}(B)$.

Corollary (11)

For any s-t flow f and any s-t cut $\text{cut}(A, B)$: $v(f) \leq c(A, B)$.

Proof. $v(f) = f^{\text{out}}(A) - f^{\text{in}}(A) \leq f^{\text{out}}(A)$ and

$$f^{\text{out}}(A) = \sum_{e \text{ out of } A} f(e) \leq \sum_{e \text{ out of } A} c_e = c(A, B).$$

Q.E.D.

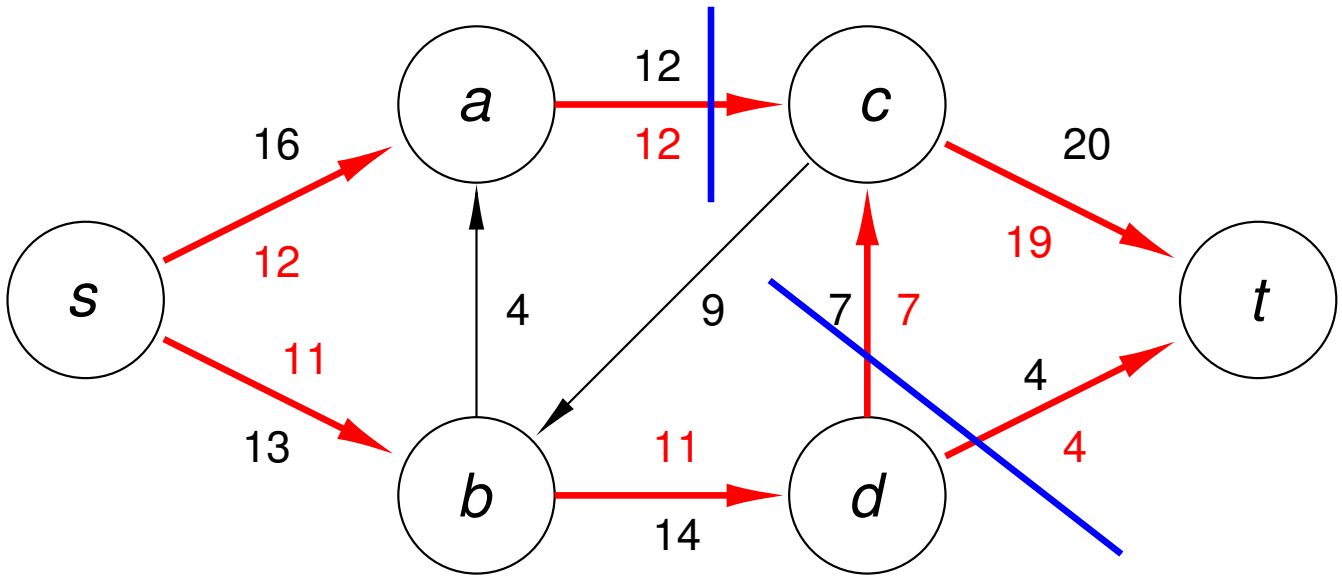
The value of every flow is bounded by the capacity of every cut.

An Iterative Algorithm for Maximum Flow

1. The Maximum-Flow Problem
   - weighted directed graphs as flow networks
   - the Ford-Fulkerson algorithm
   - termination and running time
2. Maximum Flows and Minimum Cuts
   - running a numerical example
   - flows and cuts
   - max flow equals min cut
3. Choosing Good Augmenting Paths
   - an example with a large number of iterations
   - the scaling max-flow algorithm

maximum flow equals minimal cut

On our numerical example, the minimal cut is drawn in blue where the maximum flow of 23 is reached:



`Max-Flow` returns the maximum flow

If there is no s-t path in G_f, then `Max-Flow` terminates.

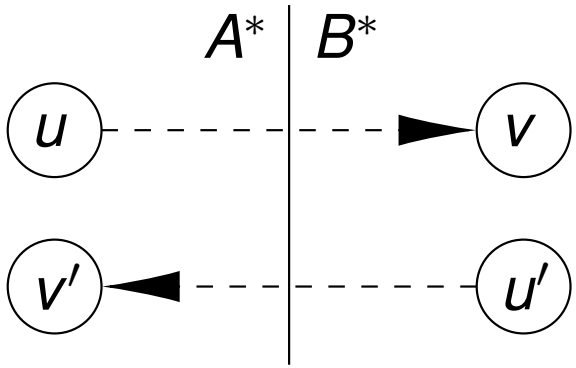
Theorem (12)

If f is an s-t flow such that there is no s-t path in G_f, then there is an s-t cut (A^*, B^*) for which $v(f) = c(A^*, B^*)$, where $v(f)$ is the maximal flow and $c(A^*, B^*)$ is the minimum capacity.

Proof. Let $A^* = \{\ v \in V \mid$ there is an s-v path in $G_f\ \}$. Set $B^* = V \setminus A^*$.

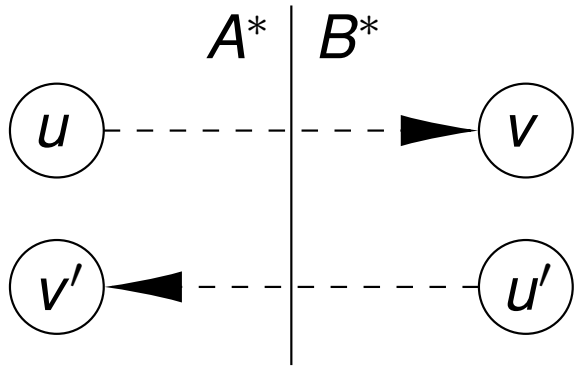
We verify that (A^*, B^*) is a cut. By construction, A^* and B^* partition V and $s \in A^*$. Because there is no s-t path in G_f, $t \in B^*$.

Consider G_f:



two claims about $f((u, v))$ and $f((u', v'))$

G_f:



$f((u, v)) = c_{(u,v)}$

$f((u', v')) = 0$

1. For any (u, v) with $u \in A^*$ and $v \in B^*$: $f((u, v)) = c_{(u,v)}$.

   By contradiction, assume $f((u, v)) < c_{(u,v)}$.
   Then (u, v) is a forward edge. As $u \in A^*$, we can extend the s-u path with v, but as $v \in B^*$, this is a contradiction.

2. For any (u', v') with $u' \in B^*$ and $v' \in A^*$: $f((u', v')) = 0$.

   By contraction, assume $f((u', v')) > 0$.
   Then (u', v') is a backward edge. As $v' \in A^*$, we can extend the s-v' path with u', but as $u' \in B^*$, this is a contradiction.

proof continued

By Lemma (10),
for any s-t flow f and any $\text{cut}(A, B)$, an s-t cut: $v(f) = f^{\text{out}}(A) - f^{\text{in}}(A)$.

$$
\begin{aligned}
v(f) &= f^{\text{out}}(A^*) - f^{\text{in}}(A^*) \\
&= \sum_{e \text{ out of } A^*} f(e) - \sum_{e \text{ into } A^*} f(e) \\
&= \sum_{e \text{ out of } A^*} f(e) - 0 \\
&= \sum_{e \text{ out of } A^*} c_e \\
&= c(A^*, B^*).
\end{aligned}
$$

Q.E.D.

optimality

Corollary (13)

The flow returned by `Max-Flow` *is a maximum flow.*

Corollary (14)

Given a flow with maximum value, we can compute an s-t cut of minimum capacity in $O(m)$ time.

Proof. We follow the proof of Theorem (12).

We construct G_f and run a breadth-first or depth-first search to compute A^*, the set of all nodes that can be reached from s. This computation runs in $O(m+n)$ time, or in time $O(m)$ as $m \geq n-1$.

The set B^* is obtained as the complement of A^* with respect to V: $B^* = V \setminus A^*$. Q.E.D.

the Max-Flow Min-Cut Theorem

Corollary (15)

In every flow network,
there is a flow f and a cut (A, B) so that $v(f) = c(A, B)$.

Theorem (16)

In every flow network, the maximum value of an s-t flow is equal to the minimum capacity of an s-t cut.

computing the minimal cut directly

Exercise 6:

Given a flow network,
design an algorithm to directly compute the minimal cut.

1. The initial cut is defined by the edges at the source, or the edges at the sink if the sum of the capacities of those edges is smaller.
2. As long as the flow on the cut cannot be realized, adjust the selection of the edges in the cut.

Elaborate the second step into an algorithm.

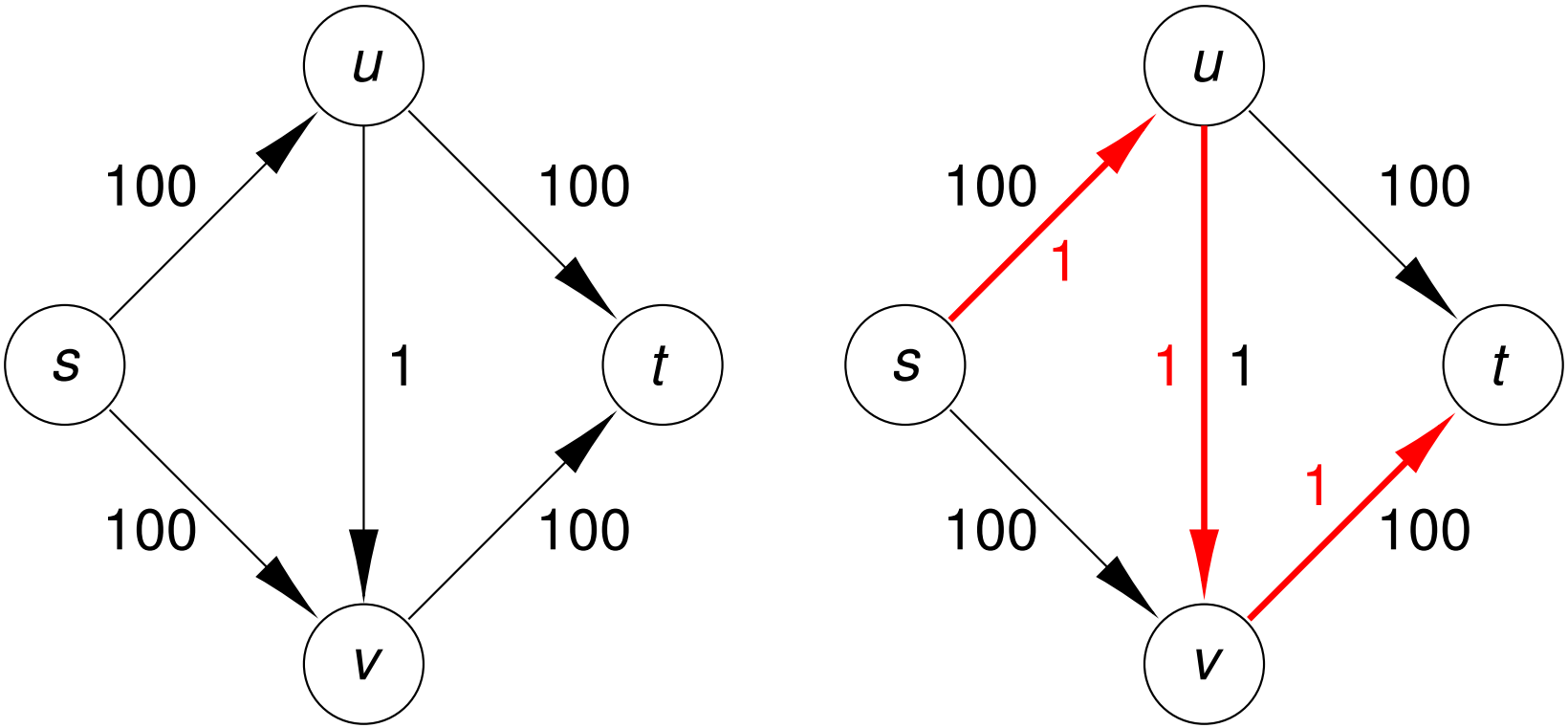
1. Prove termination and correctness.
2. Derive the asymptotic cost of this algorithm.

An Iterative Algorithm for Maximum Flow

1. The Maximum-Flow Problem
   - weighted directed graphs as flow networks
   - the Ford-Fulkerson algorithm
   - termination and running time
2. Maximum Flows and Minimum Cuts
   - running a numerical example
   - flows and cuts
   - max flow equals min cut
3. Choosing Good Augmenting Paths
   - an example with a large number of iterations
   - the scaling max-flow algorithm

an example

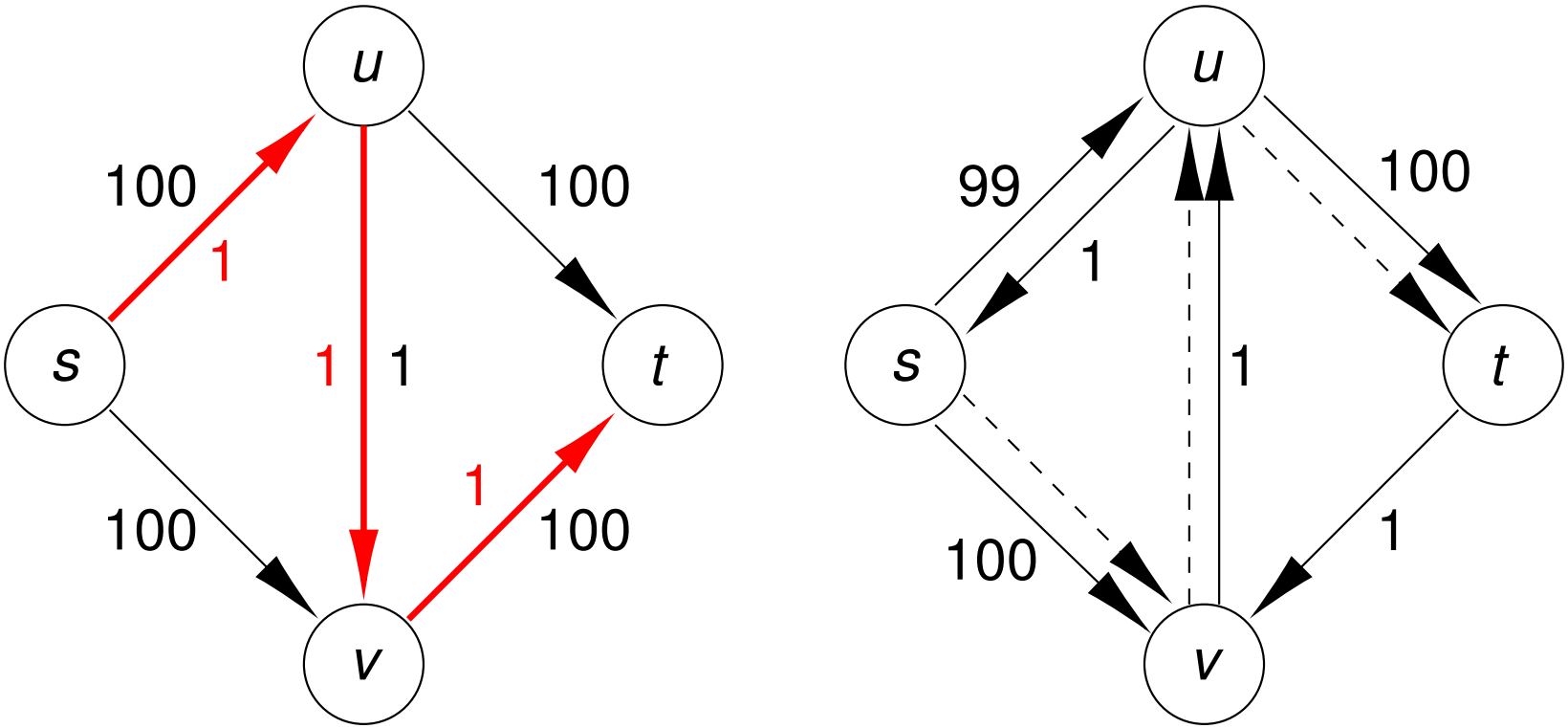
We push a flow of 1: $f((s,u)) = 1$, $f((u,v)) = 1$, $f((v,t)) = 1$:



Observe that the bound C is 200 for this network.

the residual graph

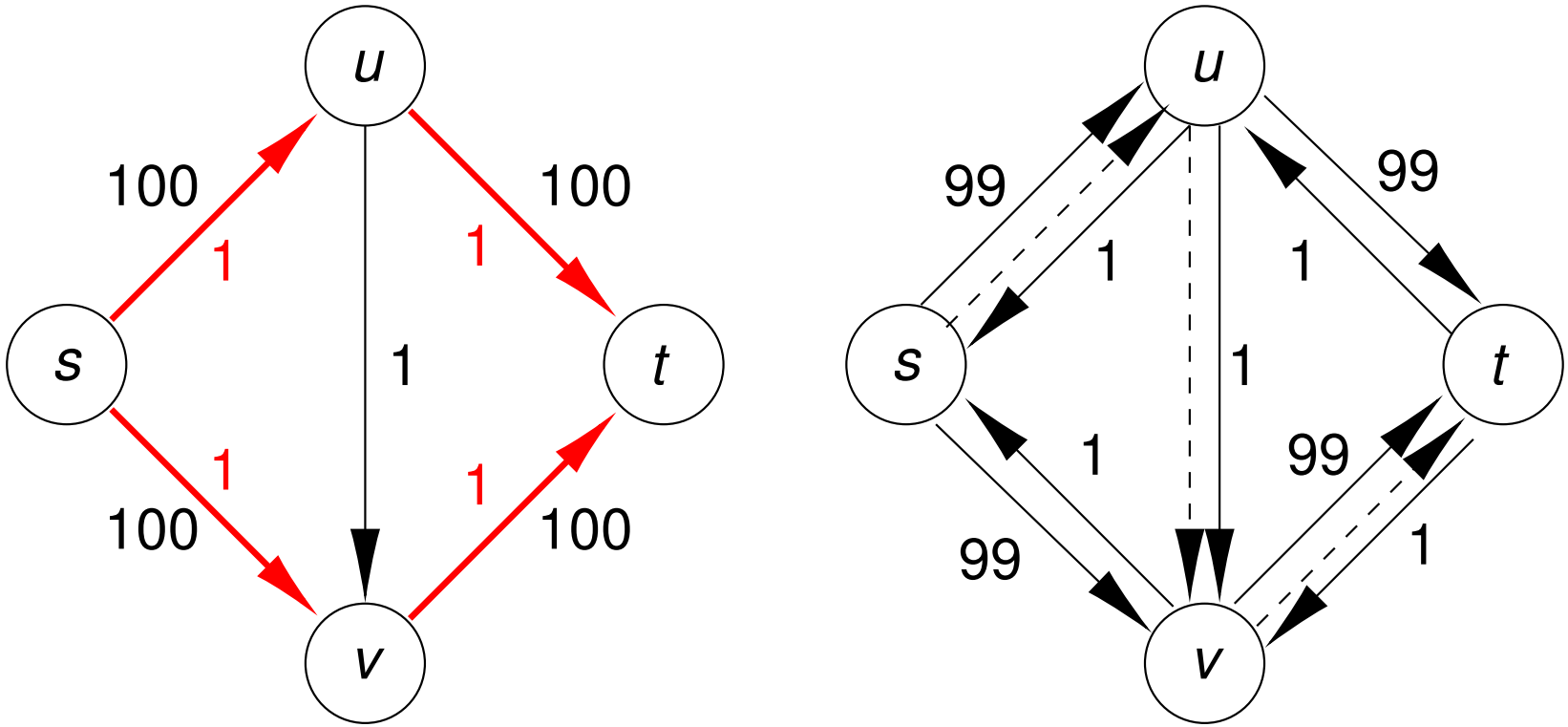
The $\texttt{bottleneck}(P, f) = 1$:



The dashed arrows indicate the augmenting path.

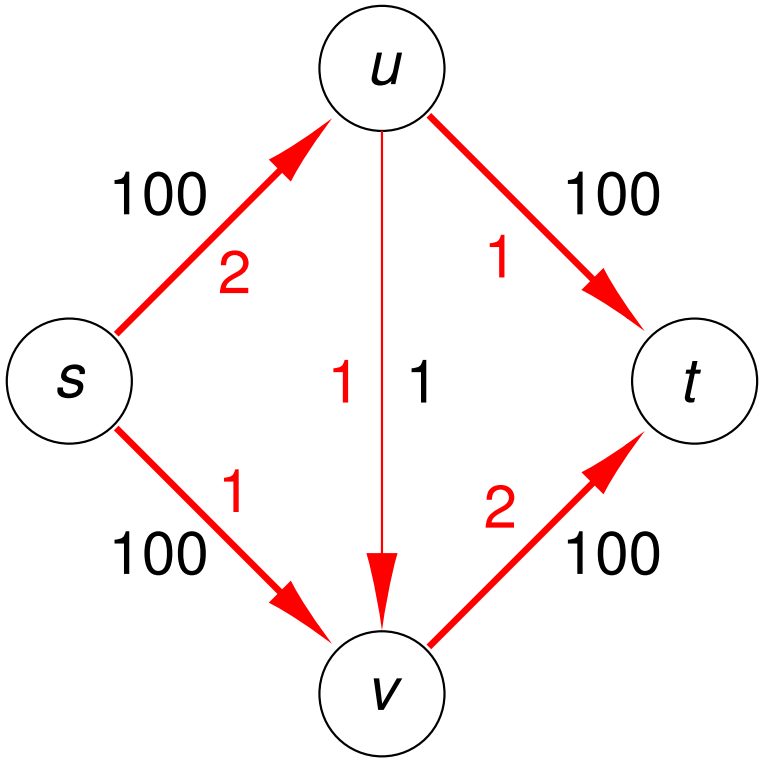
the next step

After the next step, the $\texttt{bottleneck}(P, f) = 1$ again:



The dashed arrows indicate the augmenting path.

the next steps ...



The next augmenting path will again go along (v, u).

In total 200 augmentations will be needed if in every step we choose (v, u) or (u, v) in the augmenting path.

An Iterative Algorithm for Maximum Flow

1. The Maximum-Flow Problem
   - weighted directed graphs as flow networks
   - the Ford-Fulkerson algorithm
   - termination and running time
2. Maximum Flows and Minimum Cuts
   - running a numerical example
   - flows and cuts
   - max flow equals min cut
3. Choosing Good Augmenting Paths
   - an example with a large number of iterations
   - the scaling max-flow algorithm

using a scaling parameter

We can avoid the large number of augmentations *not* considering paths that have small values of the bottleneck.

Use Δ as a *scaling parameter* and look for paths that have bottleneck capacity of at least Δ.

Denote $G_f(\Delta)$ the subset of the residual graph G_f
which contains only edges with residual capacity of at least Δ.

the scaling max-flow algorithm

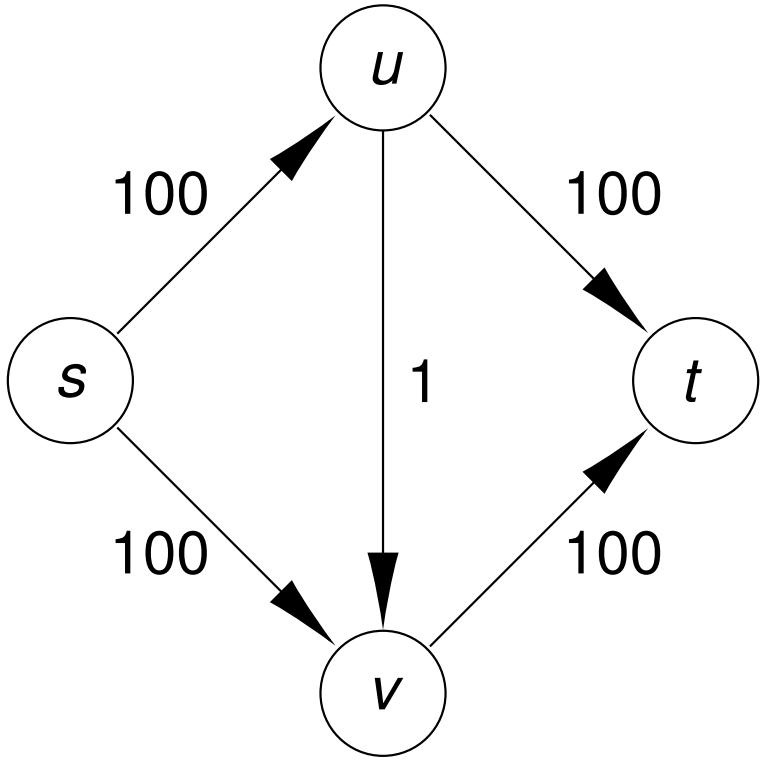
`Scaling Max-Flow`(G)

Input: $G = (V, E)$, a flow network.
Output: the maximum flow f.

for all $e \in E$: $f(e) := 0$
$C := \sum_{e \text{ out of } s} c_e$
$\Delta :=$ the largest power of 2 that is no larger than C
while $\Delta \geq 1$ do
    while there is an s-t path in $G_f(\Delta)$ do
        let P be an s-t path with no repeat nodes in $G_f(\Delta)$
        $f :=$ `augment`(f, P)
    $\Delta := \Delta/2$
return f

running on an example

Exercise 7: Consider the previous example:



Run the `Scaling Max-Flow` algorithm on this graph.
How many iterations are needed?

termination and optimality

The termination property of the `Max-Flow` algorithm extends.

Theorem (17)

If all capacities are integer numbers,
then throughout the `Scaling Max-Flow` *algorithm the flow and the residual capacities are integer numbers.*
This implies that when $\Delta = 1$, $G_f(\Delta)$ *is the same as* G_f, *and hence when the algorithm terminates the flow* f, *is of maximum value.*

The Δ-scaling phase is the outer loop of `Scaling Max-Flow`.
The value of Δ is at most C and then drops by factors of 2.
The value of Δ never gets below 1.

Lemma (18)

The number of iterations in the outer loop of `Scaling Max-Flow` *is at most* $\lceil log_2(C) \rceil$.

on the number of augmentations

During the Δ-scaling phase,
we use only edges with residual capacity of at least Δ.

Lemma (19)

During the Δ-scaling phase, each augmentation increases the flow value by at least Δ.

At the end of the Δ-scaling phase,
the flow f cannot be too far from the maximum possible value.

Theorem (20)

Let f be the flow at the end of the Δ-scaling phase.
There is an s-t $\mathrm{cut}(A, B)$ in G for which $c(A, B) \leq v(f) + m\Delta$,
where m is the number of edges in the graph G. Consequently, the maximum flow in the network has value at most $v(f) + m\Delta$.

the running time of `Scaling Max-Flow`

Lemma (21)

The number of augmentations in a scaling phase is at most $2m$.

We have at most $\lceil \log_2(C) \rceil$ scaling phases
and at most $2m$ augmentations in each scaling phase.

Theorem (22)

The `Scaling Max-Flow` *algorithm in a graph with m edges and integer capacities finds a maximum flow in at most $2m(1 + \lceil \log_2(C) \rceil)$ augmentations.*
It can be implemented to run in at most $O(m^2 \log_2(C))$ time.

iterative algorithms

The Ford-Fulkerson algorithm solves the max flow problem in a weighted directed graph. It is an interative algorithm.

As the cost of the algorithm depends on the size of the numbers, its running time is pseudo polynomial.

In the next lecture, we apply this algorithm to computing the largest matching in a bipartite graph, which is related to

- the stable matching problem, and
- the sequence alignment problem.

Many problems can be reduced to the max flow problem.